

Advanced NLP

Session 11 - Decoding Algorithms, Sampling Strategies

Teacher: Dr. Behrouz Minaei-Bidgoli
Presenter: Mohammad Khaki
Presentation creator: Presenter: Mohammad Khaki

Contents

1. Why Reasoning Matters
2. From Language Modeling to Reasoning
3. Decoding and Search Strategies

1.

Why Reasoning Matters

WHY REASONING MATTERS

Why Reasoning Matters in Modern Language Models

1. Evolution to Transformers

- Sequential Bottlenecks: Recurrent networks (RNN/LSTM) suffered from vanishing gradients and sequential processing limits.
- Attention Revolution: Self-attention and positional encodings eliminated recurrent bottlenecks, unlocking parallel context modeling.
- Pretraining Objective: Autoregressive token prediction $p(w_t \mid w_{<t})$ over web-scale text achieves remarkable linguistic fluency.
- The Direct Barrier: Standard pretraining optimizes surface token likelihood, which does not guarantee multi-step logical correctness.

2. The Reasoning Frontier

- Direct Prediction Barrier: Mapping x ➔ y in a single forward pass forces constant computational depth regardless of task complexity.
- Fluency != Correctness: Pretrained LLMs can generate mathematically flawed outputs with high confidence and grammatical elegance.
- Shift to Trajectories: Complex tasks demand intermediate computational scratchpads: x ➔ z1 ➔ ... ➔ zk ➔ y.
- Core Objective: Moving beyond pattern matching by allocating dynamic compute during inference for deliberate search and verification.

3. Course Architecture

- Session 1 (Foundations): Autoregressive decoding (MAP vs Greedy vs Beam Search), stochastic sampling (Top-k, Nucleus Top-p, Temperature), and Chain-of-Thought prompting.
- Session 2 (Search & Verifiers): Majority voting (Self-Consistency), Outcome & Process Reward Models (ORM/PRM), and test-time compute scaling laws.
- Frontier Models: Reinforcement learning on verifiable reasoning rewards (DeepSeek-R1 / GRPO), and reasoning faithfulness.
- Pedagogical Goal: Bridging token-level generation with structured problem solving.

WHY REASONING MATTERS

Two-Session Roadmap

SESSION 11

Auto awesome

Foundations & Decoding

Autoregressive generation, MAP decoding, beam search, and sampling.

SESSION 12

psychology

Reasoning & Prompting

reasoning steps, and self-consistency.

SESSION 12

verified

Search & Training

Verification, test-time compute, reasoning-model training, and failure modes.

WHY REASONING MATTERS

Why Reasoning is a Core Research Pillar (Beyond Prompting Tricks)

Memory

A. Scaling Limitations

Pre-training compute scaling faces steep resource and data availability limits.

Strategic Shift

Moving from pure parameter scaling to **Inference-Time Compute**.

Help outline

B. Open Research Questions

Path Verification

Distinguishing true reasoning steps vs. accidental right answers.

Model Distillation

Transferring reasoning capabilities to smaller, resource-constrained models.

Auto awesome

C. Efficient Intelligence

Accessible Compute

Achieving superior problem solving without massive GPU clusters.

Architectural Impact

Empowers advanced RAG, multi-step planning, and verification layers.

2.

From Language Modeling to Reasoning

FROM LANGUAGE MODELING TO REASONING

Reasoning-Based Problem Solving

input	psychology	Fact check	Task alt
Input (X)	**Intermediate Steps z1, z2...**	**Verification**	**Answer (Y)**
Initial Problem	Search & Refinement	Logical Validation	Final Output

Working Definition

Working Definition: Leveraging intermediate computational steps, search, or iterative refinement to derive solutions.

- Multi-step computational search trajectories
- Iterative state evaluation and refinement

Decomposition

Decomposition: Reasoning involves intermediate computation that supports multi-step problem solving.

- Independent test-time compute scaling
- Reasoning depth uncoupled from output length

FROM LANGUAGE MODELING TO REASONING

From Transformer to Next-Token Probabilities

Input Sequence Context Tokens → **Q, K, V** Projections → **Self-Attention** Interactions → **Transformer** FFN & Layers → **Logits** Unscaled Scores → **Softmax** Normalization → **Token Probs** Distribution

Query, Key & Value Representations

- **Q** — **Query Vector**: What the current target token is looking for in context
- **K** — **Key Vector**: What each candidate token offers to be matched against
- **V** — **Value Vector**: Actual information content contributed by each token

Attention Computation

Attention(Q,K,V)

$$A = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Candidate Next-Token Probability Distribution



Which token should the model choose as the next token?

FROM LANGUAGE MODELING TO REASONING

From Language Modeling to Inference

With an **autoregressive language model**, the joint probability of a sequence is factorized:

$$p_\theta(y_{1:T} \mid x) = \prod_{t=1}^{T} p_\theta(y_t \mid y_{<t}, x)$$

Notation Details:

Note: we'll use **y** to refer to a full sequence $\mathbf{y}_{1t}$, where each token $\mathbf{y}_t$ is conditionally predicted given past tokens **y<t** and prompt context **x**.

FROM LANGUAGE MODELING TO REASONING

From Language Modeling to Inference

$$p_\theta(y_{1:T} \mid x) = \prod_{t=1}^{T} p_\theta(y_t \mid y_{<t}, x)$$

Training

Learn the conditional distribution over tokens.

Inference

Use the learned distribution to generate output.

STEP 01
Language Models
Autoregressive sequence scoring

STEP 02
Probability Distribution
Next-token probabilities

STEP 03
Decoding
Token selection strategy

FROM LANGUAGE MODELING TO REASONING

From Tokens to Sequence Probability

Example Sequence: "I love NLP"

$$P(x1, x2, x3) = P(x1) \cdot P(x2 \mid x1) \cdot P(x3 \mid x1, x2)$$

P(x1): prob of "I"

P(x2 | x1): prob of "love" given "I"

P(x3 | x1,x2): prob of "NLP" given "I love"

Numerical Calculation: $0.8 \times 0.6 \times 0.5 = 0.24$ → **Probability of the complete sequence** (illustrative values)

Autoregressive Generation Loop

Current Context → Next-Token Prob Dist → Decoding → Selected Token → Updated Context

Sequence Probability

Multiply the conditional probabilities of all tokens in the sequence.

Next-Token Generation

Compute the next-token probability distribution conditioned strictly on current context.

Key Takeaway: "Each selected token becomes part of the context for predicting the next token."

FROM LANGUAGE MODELING TO REASONING

Decoding as an Optimization Problem

Objective Function

MAP Sequence Search

$$\hat{y} = \text{argmax}_{y \in \mathcal{Y}} p_\theta(y \mid x)$$

CHALLENGE 01

One Output Sequence

Find single most likely sequence ŷ across vocabulary

CHALLENGE 02

Enormous Output Space Y

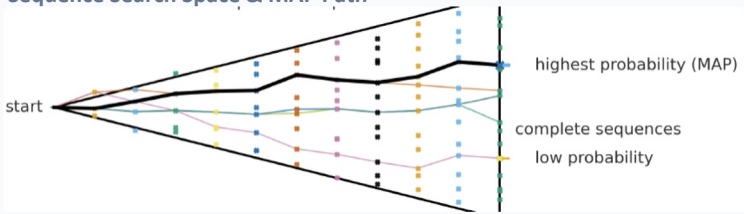
Combinatorial growth $|Y| = |V|^T$ with sequence length

CHALLENGE 03

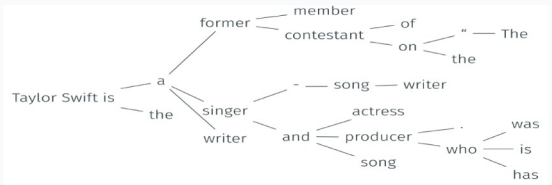
Computationally Expensive

Exhaustive search is intractable; requires heuristic search

Sequence Search Space & MAP Path



Combinatorial Prefix Tree



FROM LANGUAGE MODELING TO REASONING

Decoding as an Optimization Problem

OBJECTIVE FUNCTION

MAP Sequence Search

$$\hat{y} = \text{argmax}_{y \in \mathcal{Y}} p_\theta(y \mid x)$$

CONCEPT 01

Search Problem

Decoding is a search problem to find the single most likely output.

CONCEPT 02

Mode-Seeking

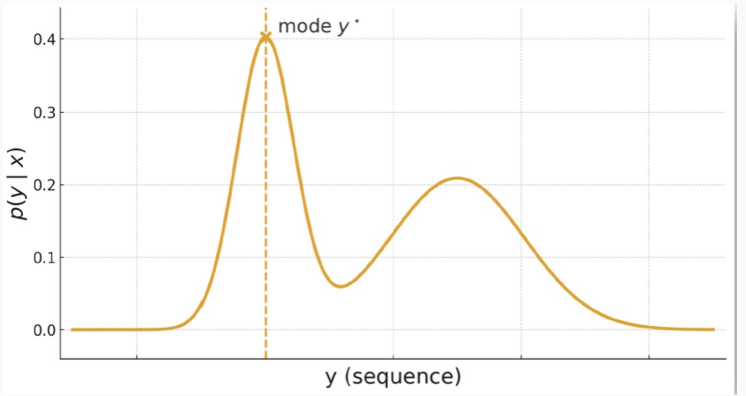
Finds a peak (mode) of the output probability distribution.

CONCEPT 03

Maximum A-Posteriori (MAP)

Given prior θ & evidence x, find a mode of the posterior.

Probability Distribution & Mode Selection



3.

Decoding and Search Strategies

DECODING AND SEARCH STRATEGIES

Greedy Decoding: Local Decisions

MAP Decoding

Argmax at Each Step

$$\hat{y} = \text{argmax}_{y \in \mathcal{Y}} p_\theta(y \mid x)$$

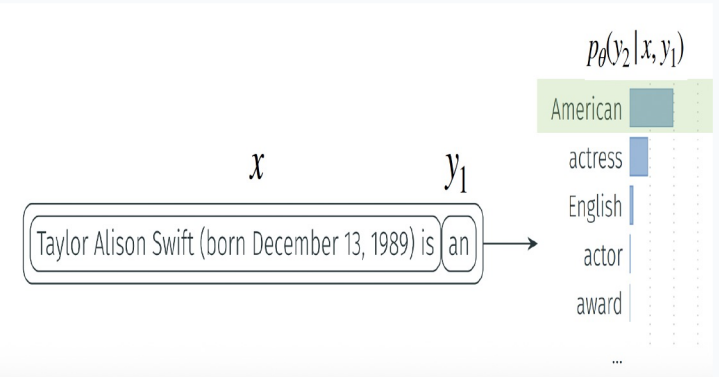
CORE QUESTION

Does choosing the best token at every step guarantee the best sequence?

Local vs. Global Optimality

Greedy search makes immediate local decisions, which can lead to sub-optimal overall sequence probability.

Step-by-Step Local Selection



DECODING AND SEARCH STRATEGIES

Greedy Decoding: Local Decisions

GREEDY SELECTION RULE

Argmax at Each Step

$$\hat{y} = \text{argmax}_{y \in \mathcal{Y}} p_\theta(y \mid x)$$

CORE QUESTION

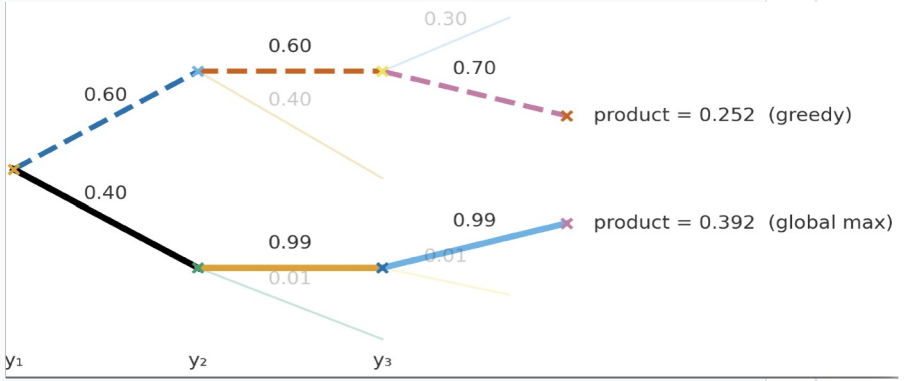
Does choosing the best token at every step guarantee the best sequence? No

Local vs. Global Optimality

Best local Choice ≠ Best global sequence

Greedy search makes immediate local decisions, which can lead to sub-optimal overall sequence probability.

Step-by-Step Path Probabilities



DECODING AND SEARCH STRATEGIES

Beam Search: Searching Multiple Paths

SEARCH STRATEGY

Width-Limited Breadth-First Search

Combines global exploration of BFS with strict memory limits.

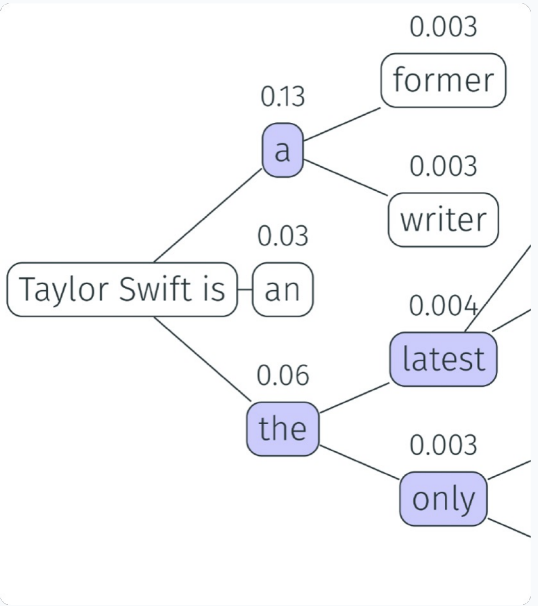
CORE IDEA

Maintain Top-B Partial Hypotheses

Beam Width (B)

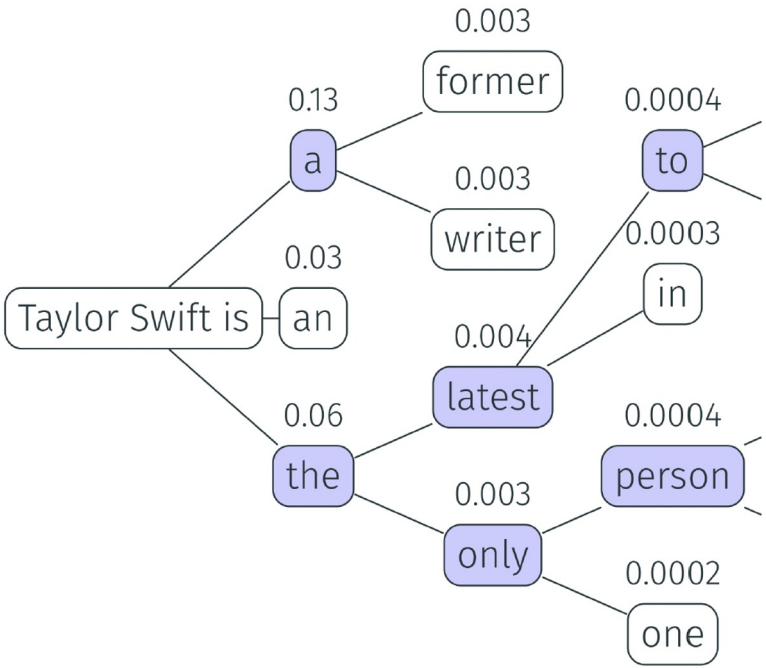
At each step, keep only the *B* most probable partial sequences (hypotheses) and prune all other paths to manage computational complexity.

Hypothesis Expansion & Search Tree



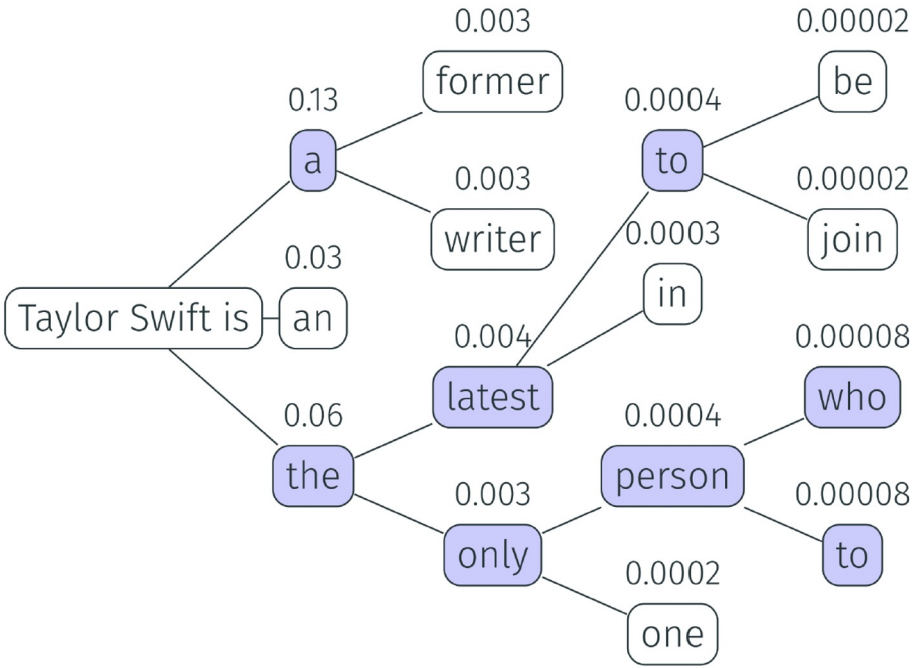
DECODING AND SEARCH STRATEGIES

Beam Search: Searching Multiple Paths



DECODING AND SEARCH STRATEGIES

Beam Search: Searching Multiple Paths



DECODING AND SEARCH STRATEGIES

Beam Search: Searching Multiple Paths

- Beam search is a width-limited breadth-first search
  - B = 1: greedy decoding
  - B = $|V|^{T_{max}}$: exact MAP
    - Example: $50000^{128} = \text{very big}$
- In practice, we use B = smaller number, e.g. 16, treated as a hyper-parameter

DECODING AND SEARCH STRATEGIES

Basic Decoding Algorithms

Greedy Search

NUMBER OF PATHS
1

SEARCH SCOPE
Local

COMPLEXITY
Lower

DIVERSITY
None

Beam Search

NUMBER OF PATHS
B (Beam Width)

SEARCH SCOPE
Multiple Hypotheses

COMPLEXITY
Higher

DIVERSITY
Limited

DECODING AND SEARCH STRATEGIES

Sampling: Exploring the Distribution

Problem 1

Beam Search

Can degenerate into repetitive loops

Problem 2

Vanilla Sampling

(Pure sampling) can quickly become incoherent

The Long-Tail Probability Dilemma

Even if most probability mass covers a limited set of good options, the tail of the distribution remains very long with many bad choices.

Although each bad token has a tiny probability, in aggregate over a sequence they have a high chance of being selected.

Sequence Risk Example

21%

Chance of staying in top 95% probability zone when generating 30 tokens in a row.

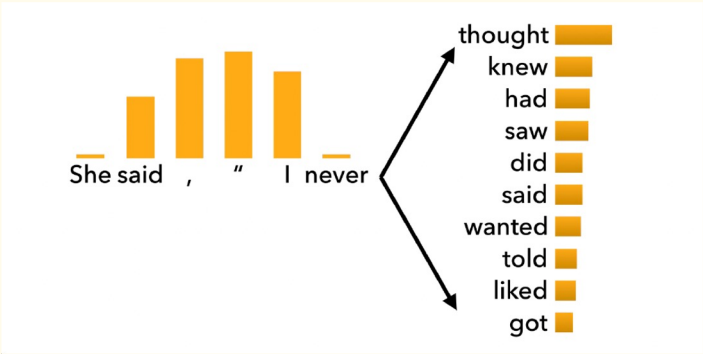
Solution

Top-k sampling (Fan et al., 2018)

Truncates the tail by sampling only from the K most likely tokens at each step.

DECODING AND SEARCH STRATEGIES

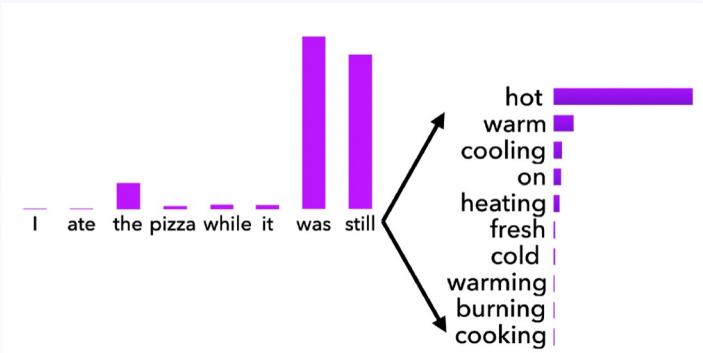
Top-K Sampling Limitations



FLAT DISTRIBUTION

May cut off too quickly!

When probability is evenly spread across many valid options, a fixed Top-*k* threshold prematurely excludes reasonable tokens.



PEAKED DISTRIBUTION

May cut off too slowly!

When one token dominates the mass, Top-*k* still forces low-probability tail tokens into the candidate set.

DECODING AND SEARCH STRATEGIES

Top-p (Nucleus) Sampling

CORE MECHANISM

Cumulative Probability Thresholding

At each step, choose the smallest candidate set whose cumulative probability is at least p, then renormalize and sample.

ADAPTIVE BEHAVIOR

Dynamic Candidate Set Size

Top-p adapts the candidate-set size dynamically:

- **Flat Distributions:** Set remains broad to preserve creative diversity.
- **Peaked Distributions:** Set becomes narrow to eliminate low-probability noise.

SAMPLING CONTROLS

Sharpness vs. Truncation

Temperature: Modifies distribution sharpness without removing tokens.

Top-k & Top-p: Directly truncate support to remove unsafe tail tokens.

These controls serve complementary, distinct purposes.

DECODING AND SEARCH STRATEGIES

Temperature Sampling

Here, you can apply temperature hyperparameter τ to the softmax to rebalance P_t :

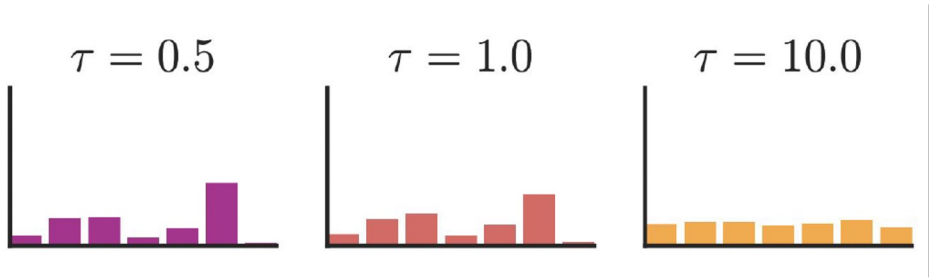
$$P_t(y_t = w|\{y_{<t}\}) = \frac{\exp(s_w/\tau)}{\sum_{w' \in V} \exp(s_{w'}/\tau)}$$

T < 1
Distribution sharper (more deterministic)

T = 1
Original distribution (unaltered)

T > 1
Distribution flatter (more random / diverse)

PROBABILITY DENSITY IMPACT



DECODING AND SEARCH STRATEGIES

Sampling, Degeneration, and Repetition

ANCESTRAL SAMPLING

Tail Accumulation

Ancestral sampling draws directly from the full next-token probability distribution.

Small tail probabilities can accumulate across long sequences, leading to degenerative text.

ERROR MODELING

Compound Error Probability

$$(1 - \varepsilon)^T$$

Probability of avoiding error for T steps

For an illustrative fixed per-step error probability ε, error-free probability decays exponentially over sequence length T.

MAP DECODING

Repetition Pitfalls

MAP-style decoding frequently produces repetitive, low-surprisal output.

Key Trade-off:

Decoding quality heavily depends on specific task goals and evaluation criteria.

Advanced NLP - Session 11 - Decoding Algorithms, Sampling Strategies – IUST

Reasoning Matters LLM to Reasoning Decoding